

Weather Condition Classification

MSML640 - Computer Vision Final Project

Akhil Shekkari · Siddharth Pathania

Classifying five weather conditions from outdoor photographs using transfer learning on EfficientNet-B0.

What We Built

A transfer-learning classifier that identifies **five weather conditions** from a single outdoor photograph, no GPS, no sensor data, just pixels.



Clear



Cloudy



Foggy



Rainy



Snowy

Why it matters: Autonomous vehicles (dashcams, ADAS), drone sensing for agriculture, and large-scale photo library tagging all benefit from robust, real-time weather awareness.

Dataset Design

Seed Pool

Public weather images from Kaggle, remapped to our five-class label scheme.

Our Photos

Self-captured College Park, reserved for val and test to introduce a deliberate **distribution shift**.

Synthetic Data

Stable Diffusion

images generated with class-specific prompts (Configs 3 & 4), stored separately to isolate their effect.

Splits

Stratified train / val / test via `prepare_data.py` and the project notebook.

Model Architecture

Backbone

ImageNet pretrained

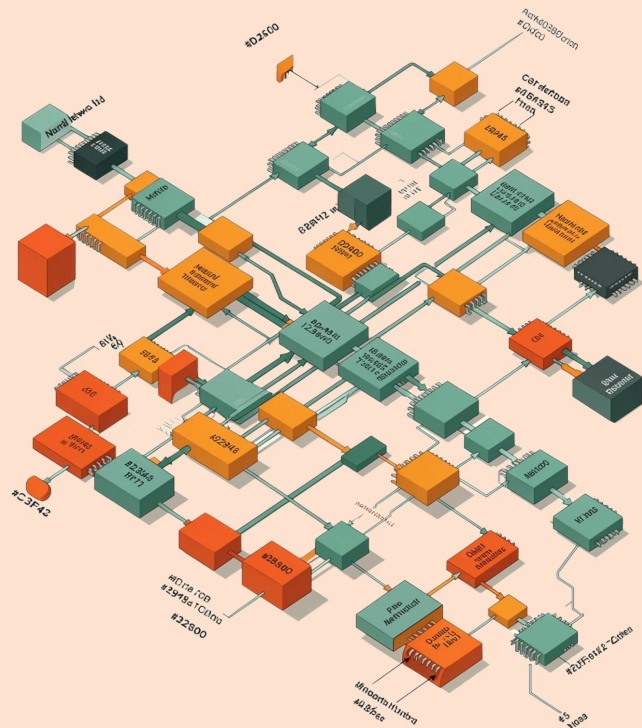
EfficientNet-B0 (~5M parameters) — compact, fast, and strong on visual features.

Training

AdamW optimizer · label smoothing **0.1** · **weighted cross-entropy** to handle class imbalance (snowy dominates; cloudy is rarer).

Environment

Google Colab GPU via `notebooks/weather_classification.ipynb`
no local setup required.



Four Ablation Configurations

Each config isolates one variable augmentation or synthetic data to measure its independent effect on performance.

Config	Training Data	Augmentation	Key Variable
1 - Baseline	Real only	None	Reference point
2 - Aug	Real only	Flip, crop, color jitter, rotation	Augmentation effect
3 - Synth	Real + Stable Diffusion	None	Synthetic data effect
4 - Synth + Aug	Real + Stable Diffusion	Same as Config 2	Combined effect

Repo & Code Structure

Notebook

`notebooks/weather_classification.ipynb` — full Colab pipeline: data prep, training, evaluation, robustness tests, and demo.

Scripts

`run_all.sh` reproduces all steps locally. Results (plots, metrics, `best_model.pt`) go to `results/`.

`dataset.py`

Transforms, class list,
dataloaders

`model.py`

EfficientNet-B0 classifier
head & weight loader

`train.py`

Training loop, weighted CE,
checkpoints

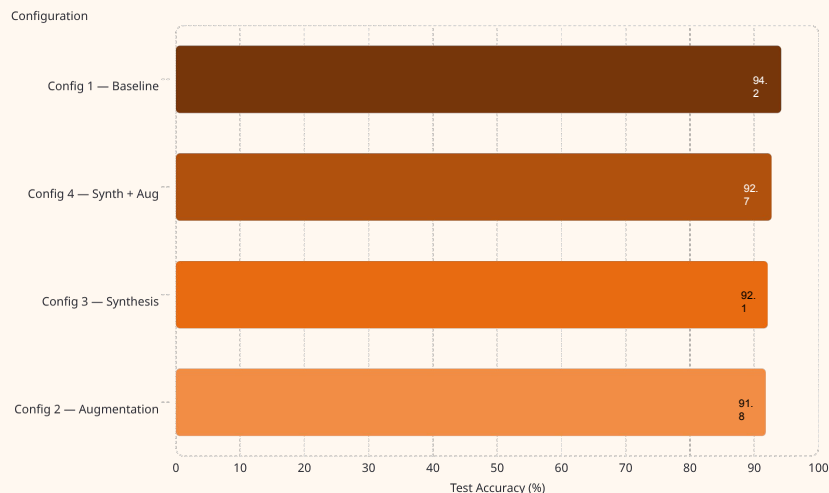
`robustness.py`

Blur, brightness, occlusion,
noise on test set

`predict.py`

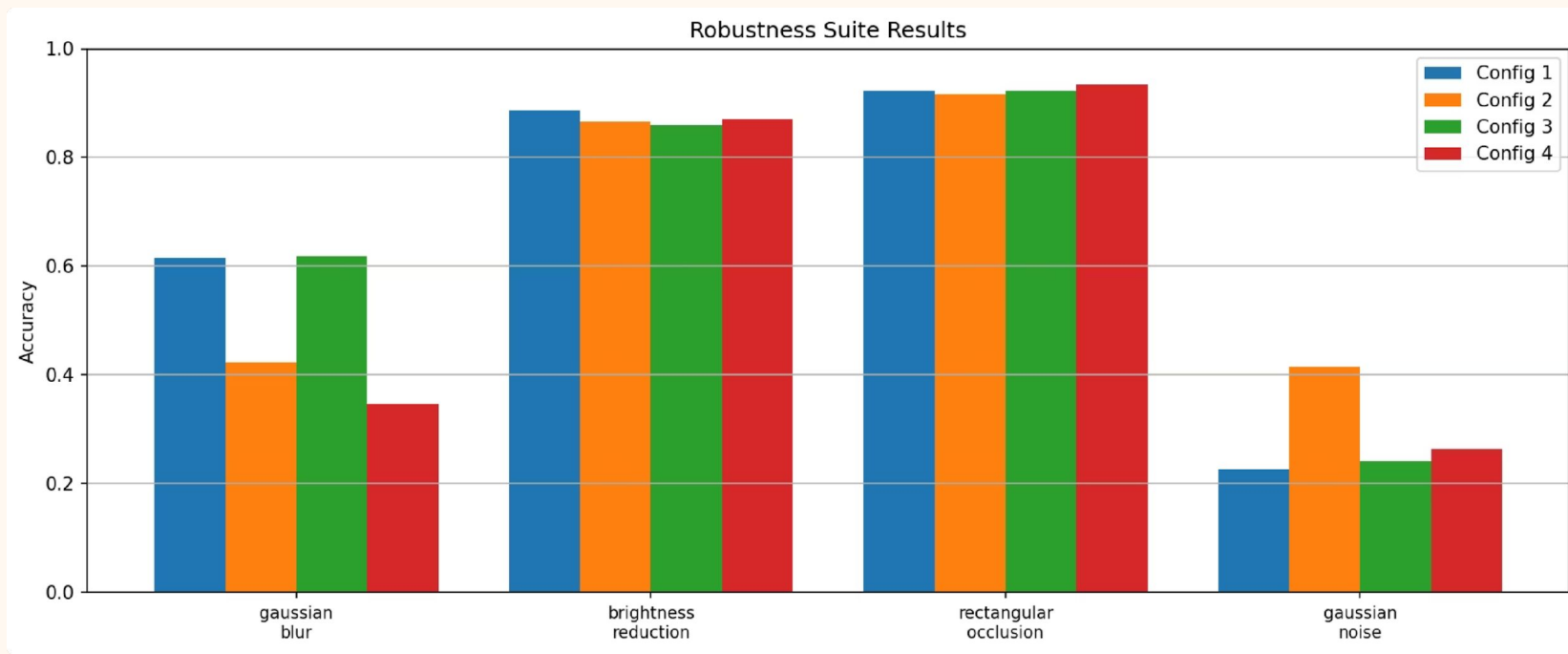
Single-image demo for new photos

Test Accuracy Results



Config 1 (Baseline) achieved the highest clean test accuracy at **94.2%** — trained for 5 epochs to fit Colab time constraints.

Robustness Highlights

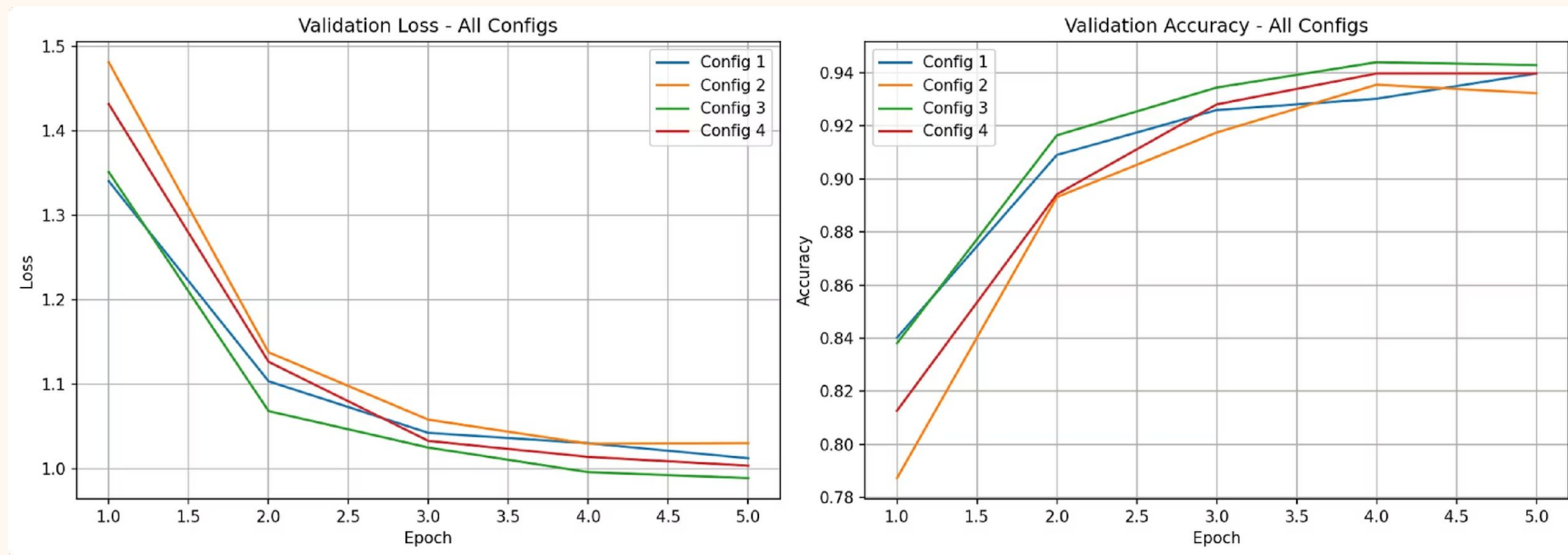


Config 2 excelled under Gaussian noise: **41.5%** vs. 22.7% baseline

Config 4 best under rectangular occlusion: **93.5%**

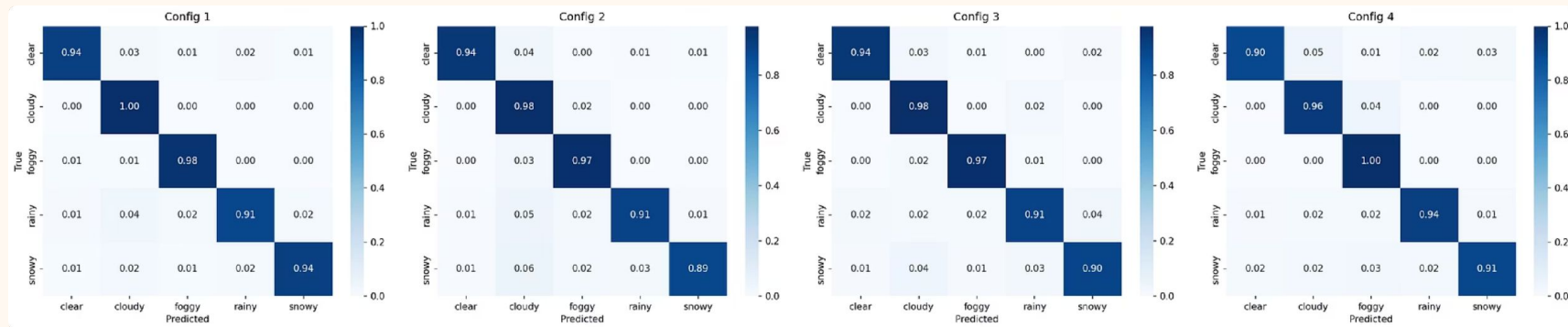
Augmentation and synthetic data trade raw accuracy for **real-world robustness**, a meaningful gain for deployment.

Validation Curves



Validation curves reveal how each configuration converges — Config 1's clean data leads to faster, sharper convergence, while augmented configs show slightly higher variance but stronger generalization under corrupted inputs.

Confusion Matrices



What to Look For

Snowy ↔ **Cloudy**: most common off-diagonal errors due to similar sky textures

Foggy ↔ **Rainy**: reduced visibility creates visual overlap

Config 2 & 4 show **better diagonal density** on minority classes

Key Takeaway

Weighted loss targeting class imbalance **improved recall on cloudy**, the rarest class, without sacrificing overall accuracy.

Synthetic data reinforced boundary cases for snowy and foggy conditions.

F1-score per class across configs:

Config	1	2	3	4
Class				
clear	0.951000	0.961000	0.947000	0.927000
cloudy	0.796000	0.662000	0.739000	0.782000
foggy	0.945000	0.923000	0.936000	0.921000
rainy	0.925000	0.916000	0.908000	0.933000
snowy	0.963000	0.938000	0.937000	0.946000

Working demo examples

Prediction for: `/content/drive/MyDrive/Computer vision final project/data/demo/Test 1.jpg`

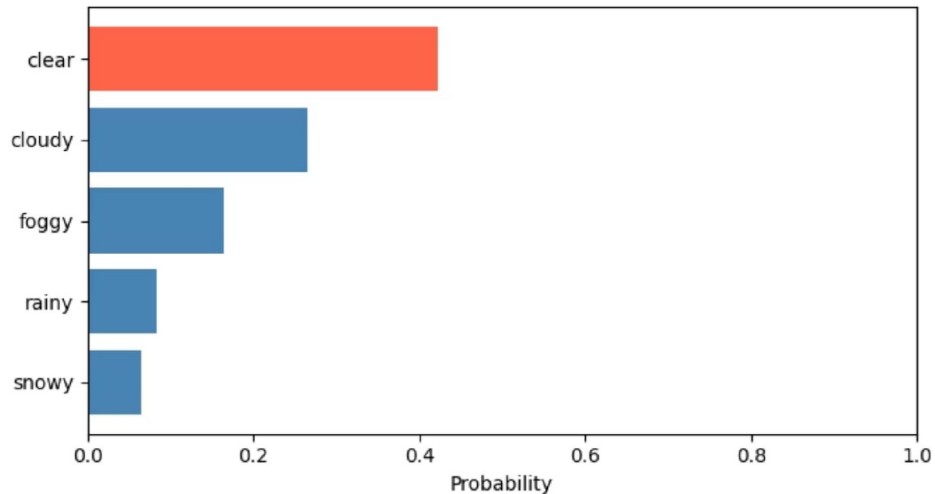
```
clear    0.423 #####  
cloudy   0.265 #####  
foggy    0.164 #####  
rainy    0.084 ###  
snowy    0.064 ##
```

Top prediction: clear

Input image



Prediction: clear (42.3%)



Prediction for: </content/drive/MyDrive/Computer vision final project/data/demo/test2.jpg>

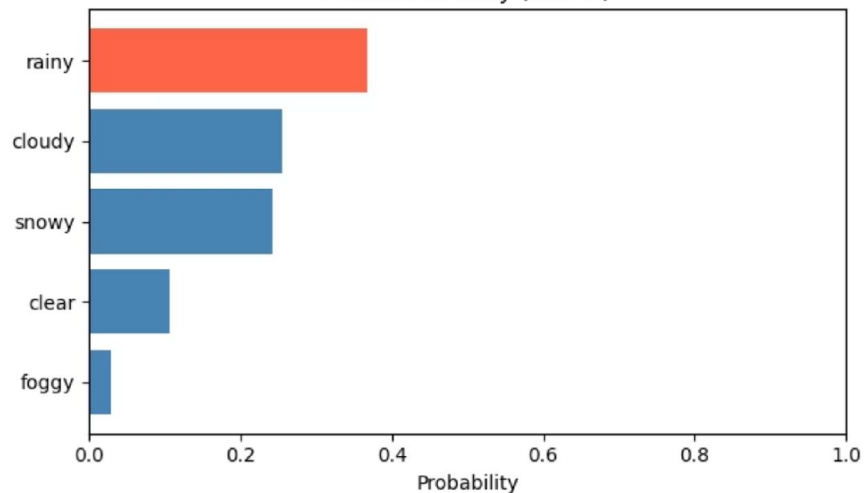
rainy	0.367	#####
cloudy	0.255	#####
snowy	0.243	#####
clear	0.107	###
foggy	0.028	#

Top prediction: rainy

Input image



Prediction: rainy (36.7%)



Key Takeaways

1 Transfer learning is powerful

EfficientNet-B0 with ImageNet weights reached **94.2%** clean accuracy in just 5 epochs — a strong baseline for a 5-class task.

2 Augmentation wins on robustness

Config 2 nearly **doubled** noise robustness (41.5% vs. 22.7%), showing augmentation is critical for deployment in real-world conditions.

3 Synthetic data adds targeted coverage

Stable Diffusion images boosted occlusion robustness in Config 4 (**93.5%**), demonstrating that generative augmentation fills distribution gaps.

Limitations and Next Steps

1 Extend Training Epochs

Further training may help configurations 2 and 4 match baseline accuracy by allowing more comprehensive learning over more epochs.

3 Improve Synthetic Data Quality

Utilizing richer prompts or fine-tuning the Stable Diffusion model could generate higher quality synthetic images, potentially improving generalization.

2 Address Cloudy Class Mismatch

The domain mismatch from a secondary dataset source continues to impact the 'Cloudy' class performance, suggesting a need for more diverse or tailored data for this category.

4 Integrate Noise Augmentation

Implementing mild noise augmentation directly during training can further enhance the model's inherent robustness to noisy real-world inputs.